Curs 4 – Charts

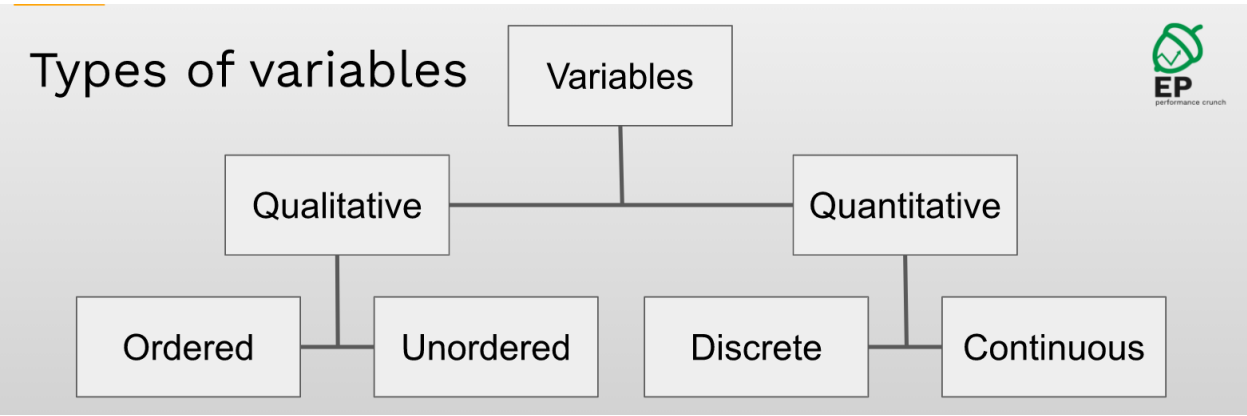
The aim of performance analysis

1. What is the aim of every performance analysis?
   Help in making a decision
2. **Presentation of final results** - one of the important steps in every performance evaluation study

Why use graphs

1. Save readers' time
   a. Present the same information more concisely
2. Interest the reader
   a. Easier to look at the figures to quickly grasp the main points of the study and read the text only for details
3. Emphasize or clarify a point
4. Reinforce a conclusion
5. Summarize the results of a study

Types of variables



- **Ordered** - Computer type: supercomputers, minicomputers, microcomputers
- **Unordered** - Workload type: scientific, engineering, educational
- **Discrete** (Integer) - Number of processors in a multiprocessor system. Size of disks in blocks.
- **Continuous** (Real) - System response time for a job.

The type of chart to be used depends on the type of variable

1. A **line** chart is used to show the ***relationship*** between two continuous **variables**.

2. A **column** chart or bar chart is used when the ***independent*** variable (also called x-variable) is a **discrete** or a **qualitative** variable.

Common chart types

- Line chart
    - Usually shows change over time
    - Compare things over time
- Bar chart
    - Compare similar things in a single category
- Double bar chart
    - Compare data
- Histogram
    - Type of bar chart
    - Compares ranges of data in a single category
- Stennplots
    - Like histograms, except they show each data point
- Pie chart
    - Shows parts of a whole
    - Often shows percentages
- Scatter plot
    - Dots of information that are not connected
    - Often show a correlation
- Timeplot
    - Shows how a variable changes over time
    - Convention
        - time on OX axis
        - values of the variable on the OY axis
- Box and whisker
    - Show variation among your data

Guidelines for good graphs

1. Require minimum effort from the reader
    a. Legends vs direct labeling
2. Maximise information - self contained
    a. CPU usage vs Daily CPU usage
3. Minimise the amount printed
    a. Grid lines - shown only if necessary to read accurate values
4. Use commonly accepted practices
    a. What people expect: origin in (0,0), linear scales, increase left to right, etc.
5. Avoid ambiguity

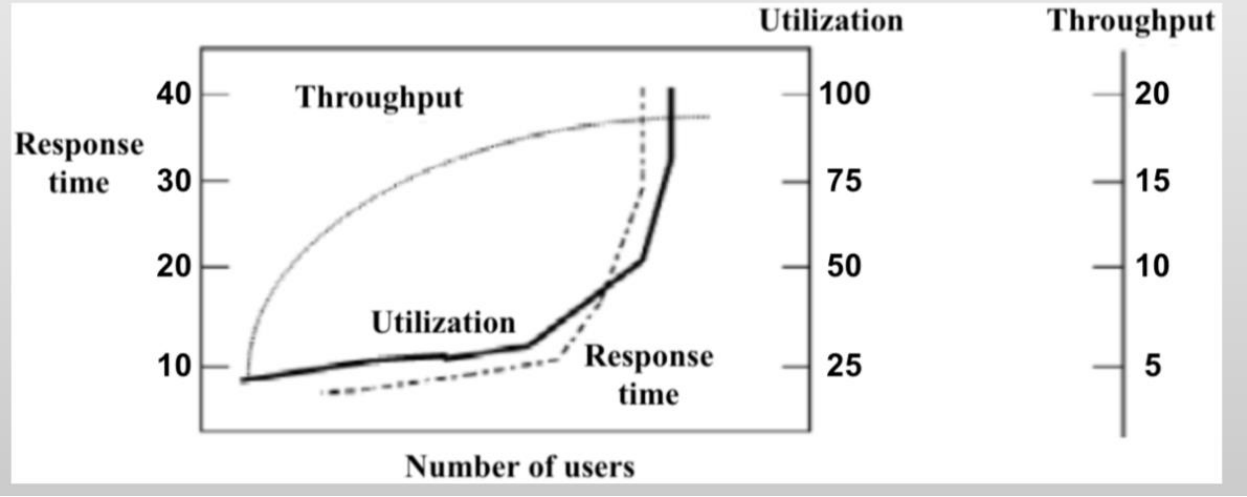
a. Show coordinate axes, scale divisions, and origin.

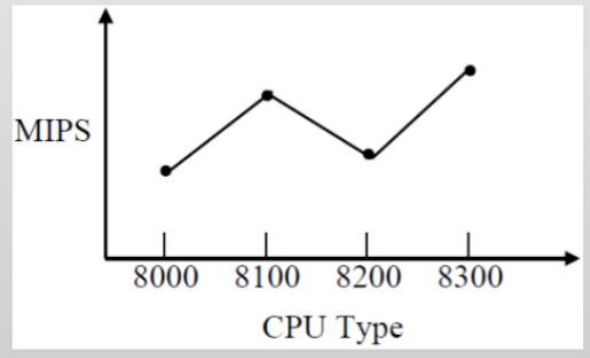
Checklist for good graphics

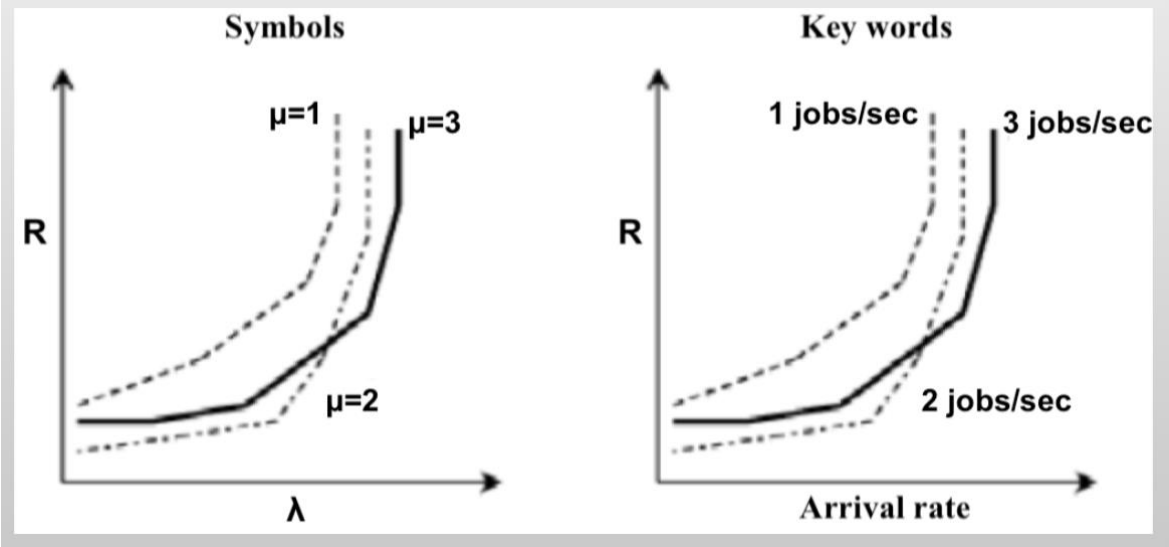
1. Are both coordinate axes shown and labeled?
2. Are the axes labels self-explanatory and concise?
3. Are the scaLes and divisions shown on both axes?
4. Are the minimum and maximum of the ranges shown on the axes appropriate to present the maximum information?
5. Is the number of curves reasonably small? (A line chart should generally have no more than six curves).
6. Do all graphs use the same scale? (Multiple scaLes on the same chart are confusing.)
7. Is there no curve that can be removed without reducing the information?
8. Are the curves on a line chart individually labeled?
9. Are the cells in a bar chart individually labeled?
10. Are all symbols on the graph accompanied by appropriate textual explanations?
11. If the curves cross, are the line patterns different to avoid confusion?
12. Are the units of measurement indicated?
13. Is the horizontal scale increasing from left to right?
14. Is the vertical scale increasing from bottom to top?
15. Are the grid lines aiding in reading the curves? (If not, the grid lines should not be shown.)
16. Does this whole chart add to information available to the reader?
17. Are the scales contiguous? (Breaks in the scale should be avoided or clearly shown.)
18. Is the order of bars in a bar chart systematic? (Alphabetic, temporal, or best-to-worst ordering is to be preferred over random placement.)
19. If the vertical axis represents a random quantity, are confidence intervals shown?
20. If the vertical axis represents a random quantity, are confidence intervals shown?
21. Are there no curves, symbols, or texts on the graph that can be removed without affecting the information? 21. Is there a title for the whole chart?
22. Is the chart title self-explanatory and concise?
23. Do the variables plotted on this chart give more information than other alternatives?
24. Does the chart clearly bring out the intended message?
25. Is the figure referenced and discussed in the text of the report?

Common mistakes

1. Too much data on a graph
    a. Rule of thumb
        i. Max: **6 lines**/curves on a line chart, **8 bars** in a bar/column chart, **8 slices** in a pie chart
        ii. Max **10 values** on an axis
2. Using symbols in place of text
3. Using line chart instead of column chart



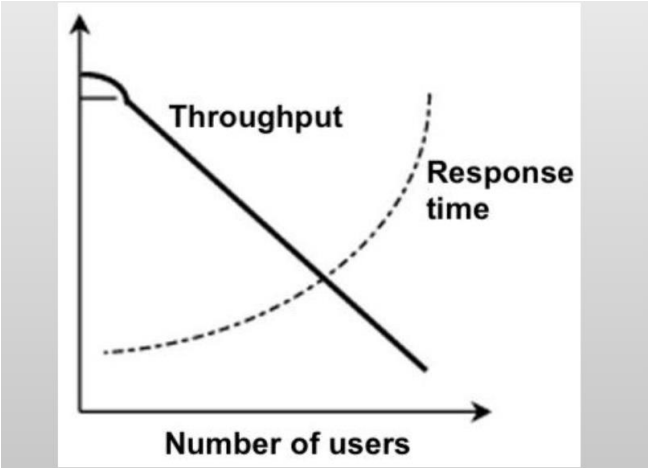


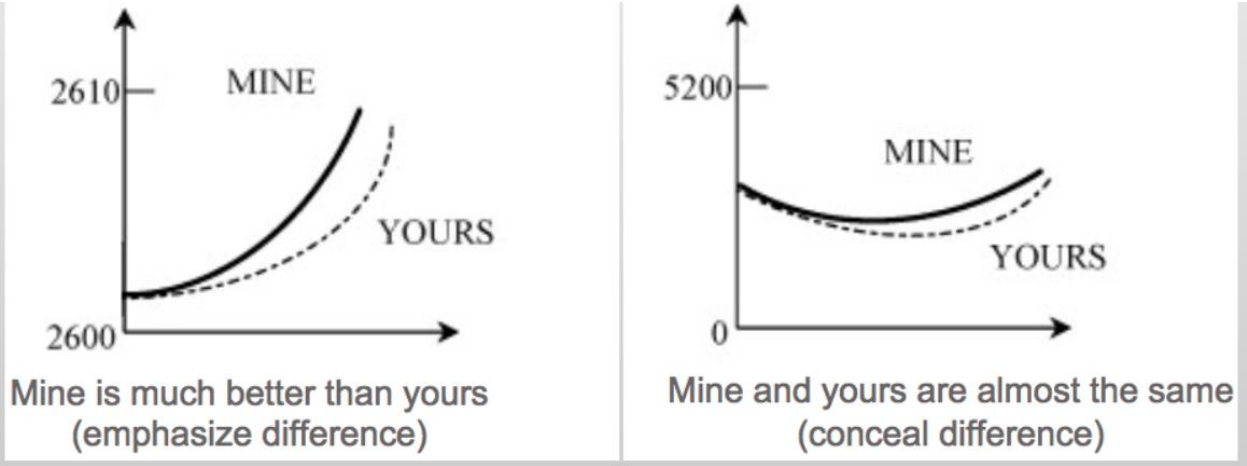


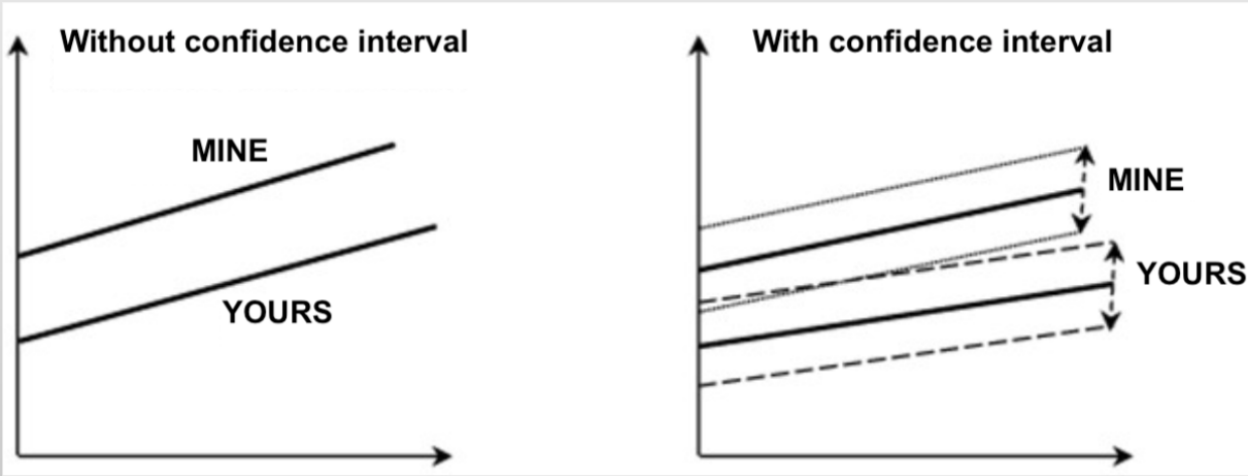
Misleading graphs

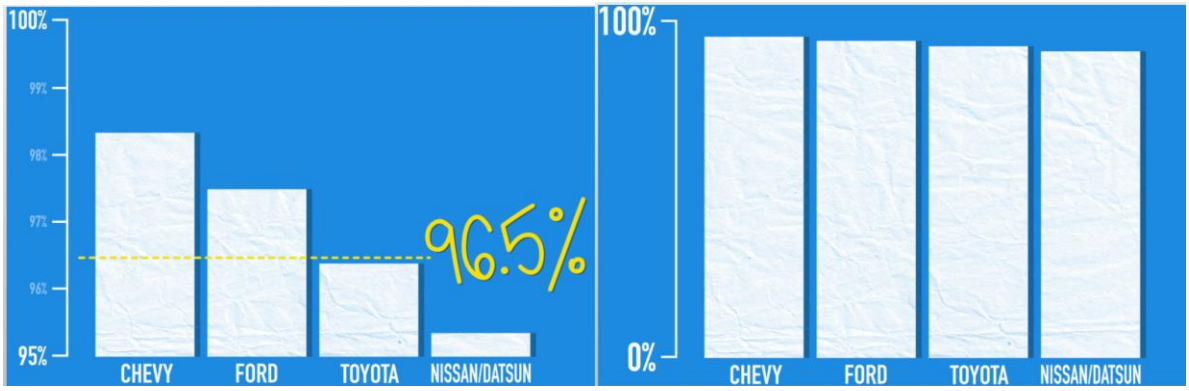
1. Using non-zero origins to emphasize the difference
2. Improper scaling
3. Plotting related metrics for 'dramatisation'

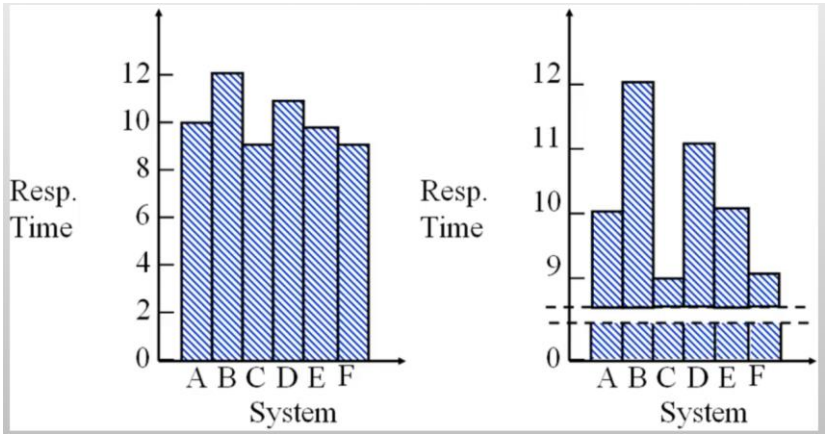
4. Omitting data
5. Improper extraction
6. Labels, numbers, scale, context



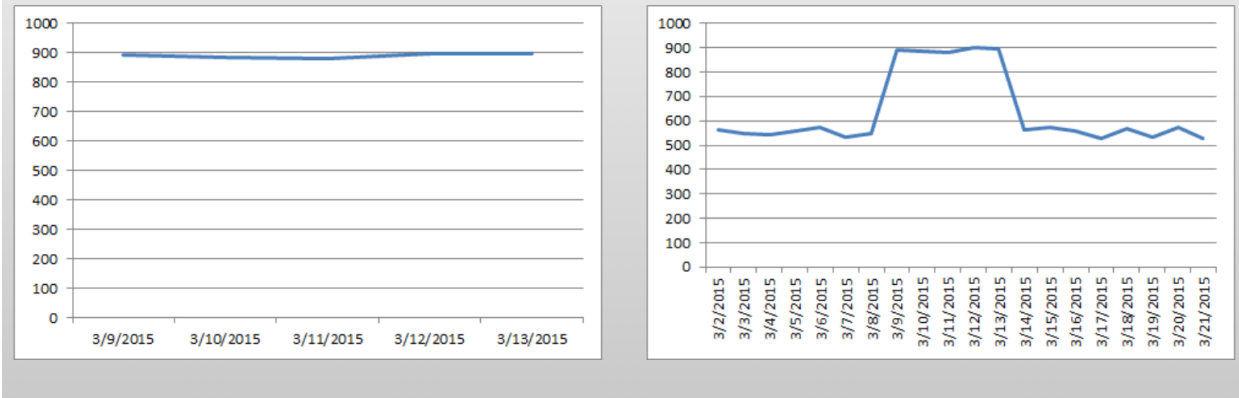


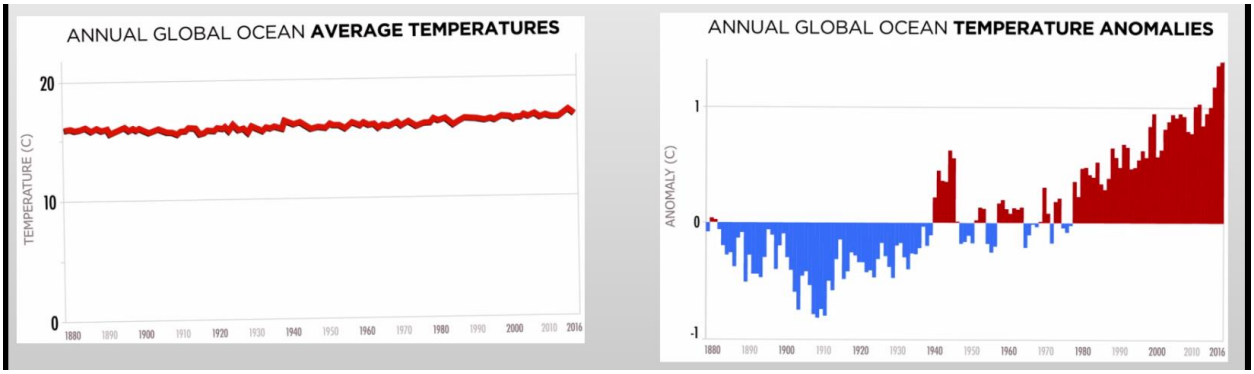


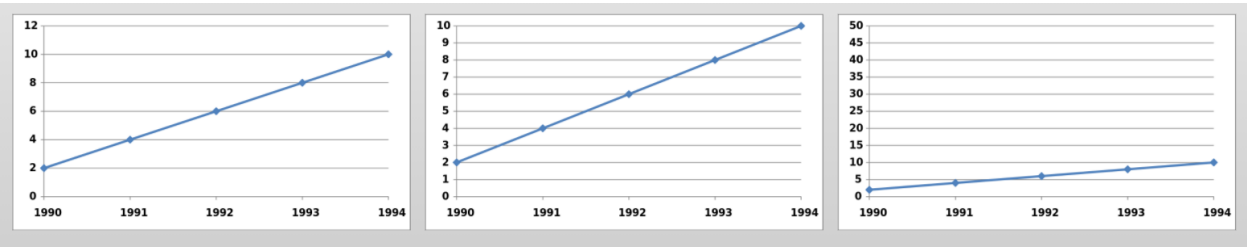

100%
99%
98%
97%
96%
95%
96.5%
CHEVY
FORD
TOYOTA
NISSAN/DATSUN
100%
0%
CHEVY
FORD
TOYOTA
NISSAN/DATSUN

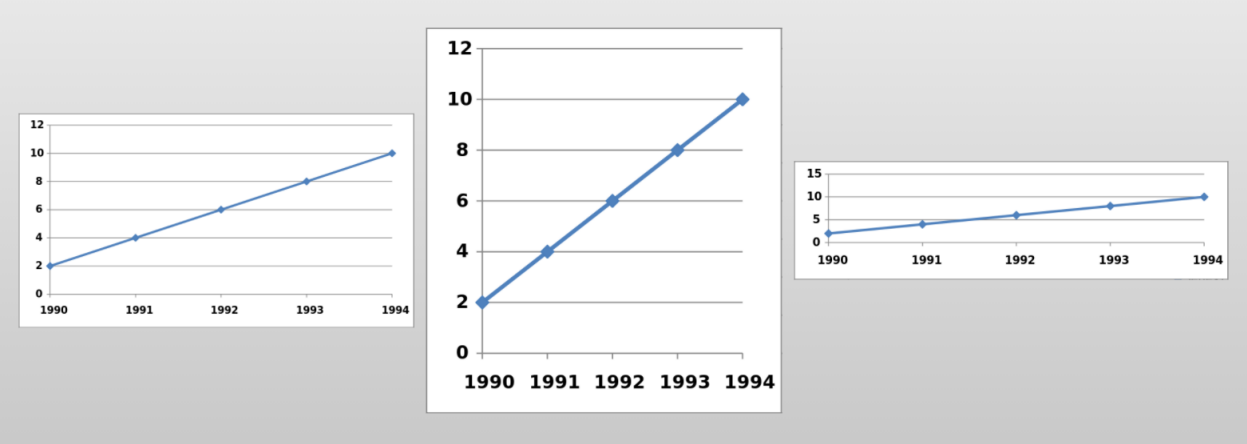

Resp. Time
12
10
8
6
4
2
0
A B C D E F
System
Resp. Time
12
11
10
9
0
A B C D E F
System

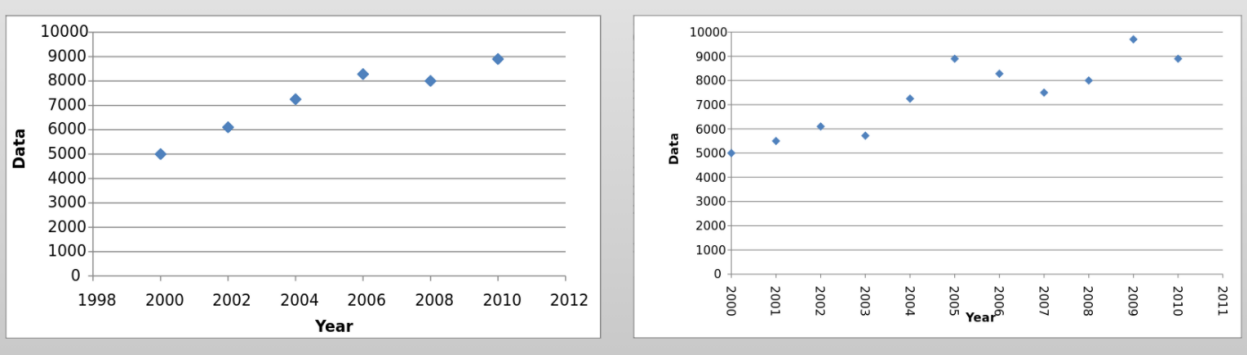

JOB LOSS BY QUARTER
15 MIL
13.5 MIL
9 MIL
7 MIL
DEC '07
SEPT '08
MAR '09
JUN '10
6
15
JOB LOSS BY QUARTER
15 MIL
7 MIL
DEC '07
SEPT '08
MAR '09
JUN '10

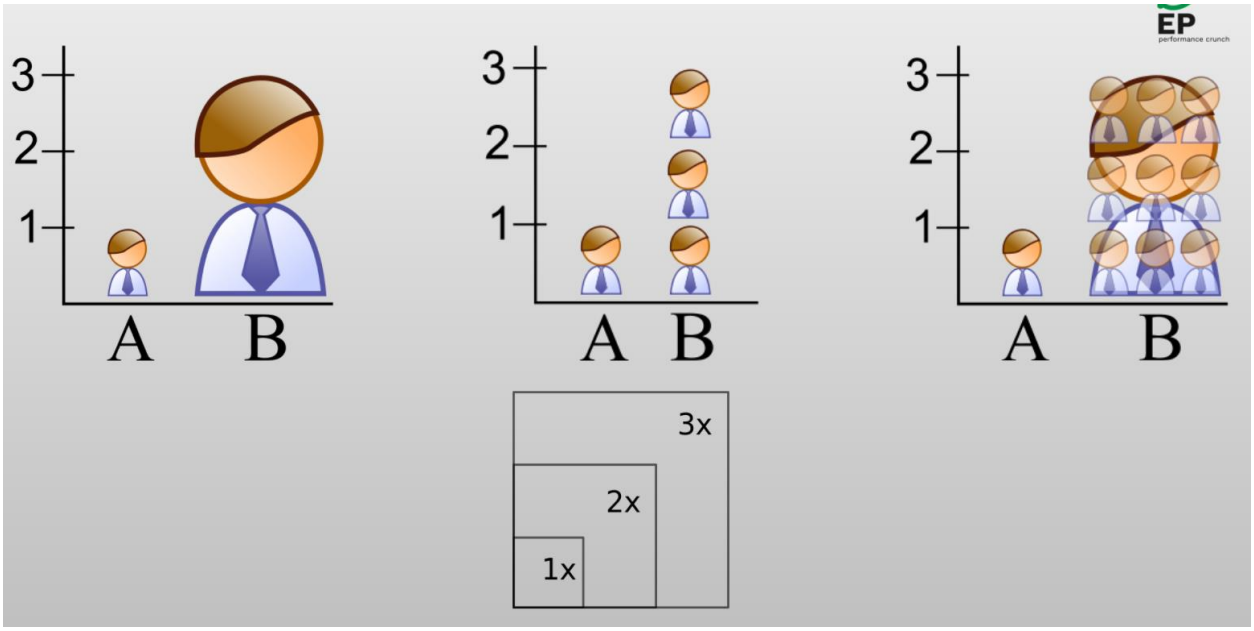

1000
900
800
700
600
500
400
300
200
100
0
3/9/2015
3/10/2015
3/11/2015
3/12/2015
3/13/2015
1000
900
800
700
600
500
400
300
200
100
0
3/2/2015
3/3/2015
3/4/2015
3/5/2015
3/6/2015
3/7/2015
3/8/2015
3/9/2015
3/10/2015
3/11/2015
3/12/2015
3/13/2015
3/14/2015
3/15/2015
3/16/2015
3/17/2015
3/18/2015
3/19/2015
3/20/2015
3/21/2015


ANNUAL GLOBAL OCEAN AVERAGE TEMPERATURES
TEMPERATURE (C)
20
10
0
1880 1890 1900 1910 1920 1930 1940 1950 1960 1970 1980 1990 2000 2010 2016
ANNUAL GLOBAL OCEAN TEMPERATURE ANOMALIES
ANOMALY (C)
1
0
-1
1880 1890 1900 1910 1920 1930 1940 1950 1960 1970 1980 1990 2000 2010 2016


12 10 8 6 4 2 0
1990 1991 1992 1993 1994
10 9 8 7 6 5 4 3 2 1 0
1990 1991 1992 1993 1994
50 45 40 35 30 25 20 15 10 5 0
1990 1991 1992 1993 1994


12 10 8 6 4 2 0
1990 1991 1992 1993 1994
12 10 8 6 4 2 0
1990 1991 1992 1993 1994
15 10 5 0
1990 1991 1992 1993 1994


Data
10000 9000 8000 7000 6000 5000 4000 3000 2000 1000 0
1998 2000 2002 2004 2006 2008 2010 2012
Year
Data
10000 9000 8000 7000 6000 5000 4000 3000 2000 1000 0
2000 2001 2002 2003 2004 2005 2006 2007 2008 2009 2010 2011
Year



Overview

- Importance
- Guidelines / best practices
- Matplotlib – Why, Intro, More details

Plotting tools

What is a plotting tool? A plotting tool is computer software which helps to analyze and visualize data, often of a scientific nature. Using this type of software, users can generate plots of functions, data and data fits.

Software of this nature typically includes additional functionality, such as data analysis functions including curve fitting.

Plotting tools

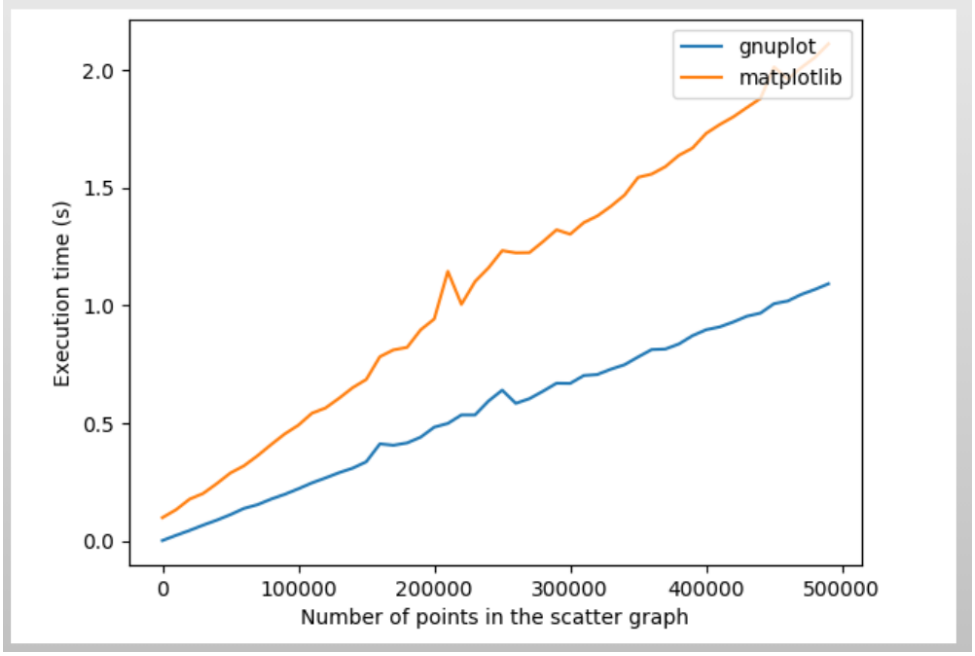
1. Online / Offline
2. Operating systems (including mobile OSs)
3. Open source / Closed source
4. Free / Paid
   Over 100 alternatives here https://en.wikipedia.org/wiki/List_of_information_graphics_software

Free, linux-based:

- **gnuptot** - Command-line driven interactive plotting program
- **Ctioga** - Command-line plotting utility

- **LabPlot** - Data plotting and function analysis tool for KDE
- **matplotlib** - Python based plotting system in a style similar to Matlab
- **Qtilnot** - Fully fledged plotting software
- **ROOT** - Solves the data analysis challenges of high-energy physics
- **SCaVis** - Produces professional graphics and has many libraries for data manipulation
- **GLE** - Graphics Layout Engine
- **PLplot** - Scientific plotting library
- **RLPLot** - Generate publication quality graphs
- **Genius** - Educational tool in mathematics and for research

Gnuplot vs. Matplotlib



Matplotlib

- **Price** - Free to download
- **Language** - Python
- **License** - Open source
- **Developer** - NumFOCUS - nonprofit organisation in the US
- **Website** - https://matplotlib.org
- **Support** - Documentation

Gnuplot

- A comprehensive library for creating static, animated, and interactive visualizations in Python
- Large user base > great documentation > actively maintained > gained popularity in recent years

Matplotlib

- MATLAB-like interface
- Low level plotting backend for virtually all plotting tools in python
    - Seaborn, pandas plotting etc.
- De-facto standard for data visualizations in data science / machine learning communities
- Interactive (ipython) or in python scripts
- 2D, 3D
- Control over all visual aspects of a plot
- Output file formats
    - PNG, EPS, SVG, JPEG, ...

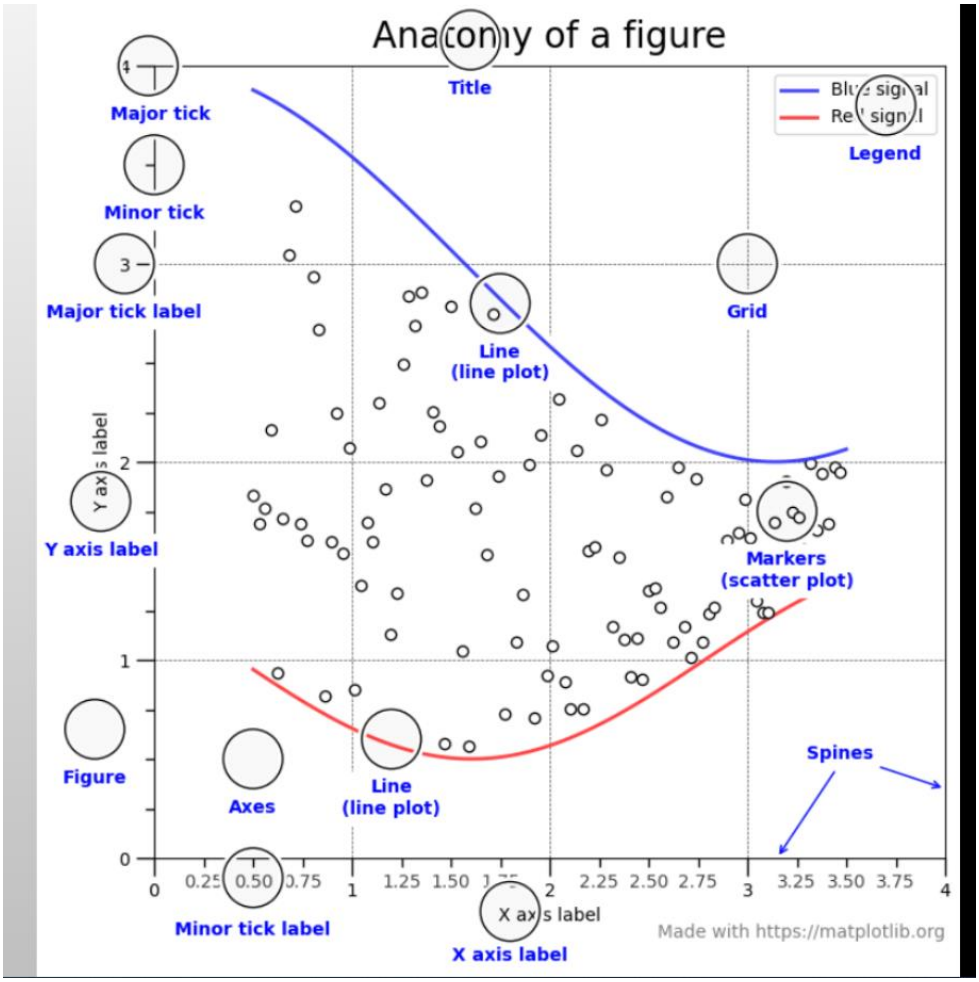
Starting up

Install matplotlib - $ pip install matplotlib

Import matplotlib - >> import matplotlib.pyplot as plt

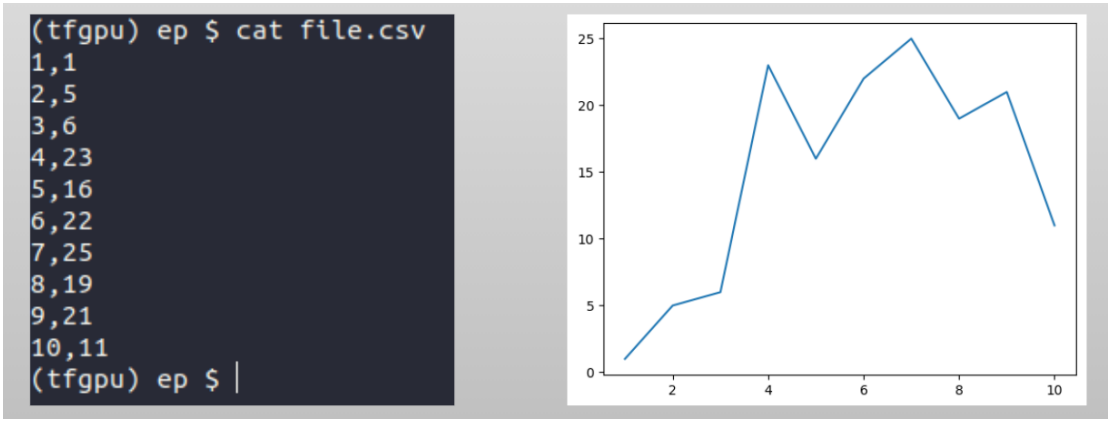
Anatomy of a figure

1. All aspects customizable
2. Axes
    a. Ranges, major / minor ticks, labels
3. Title
4. Markers
    a. Size, color, shape
5. Lines
    a. Thickness, colors, types
6. Grid
7. Legend

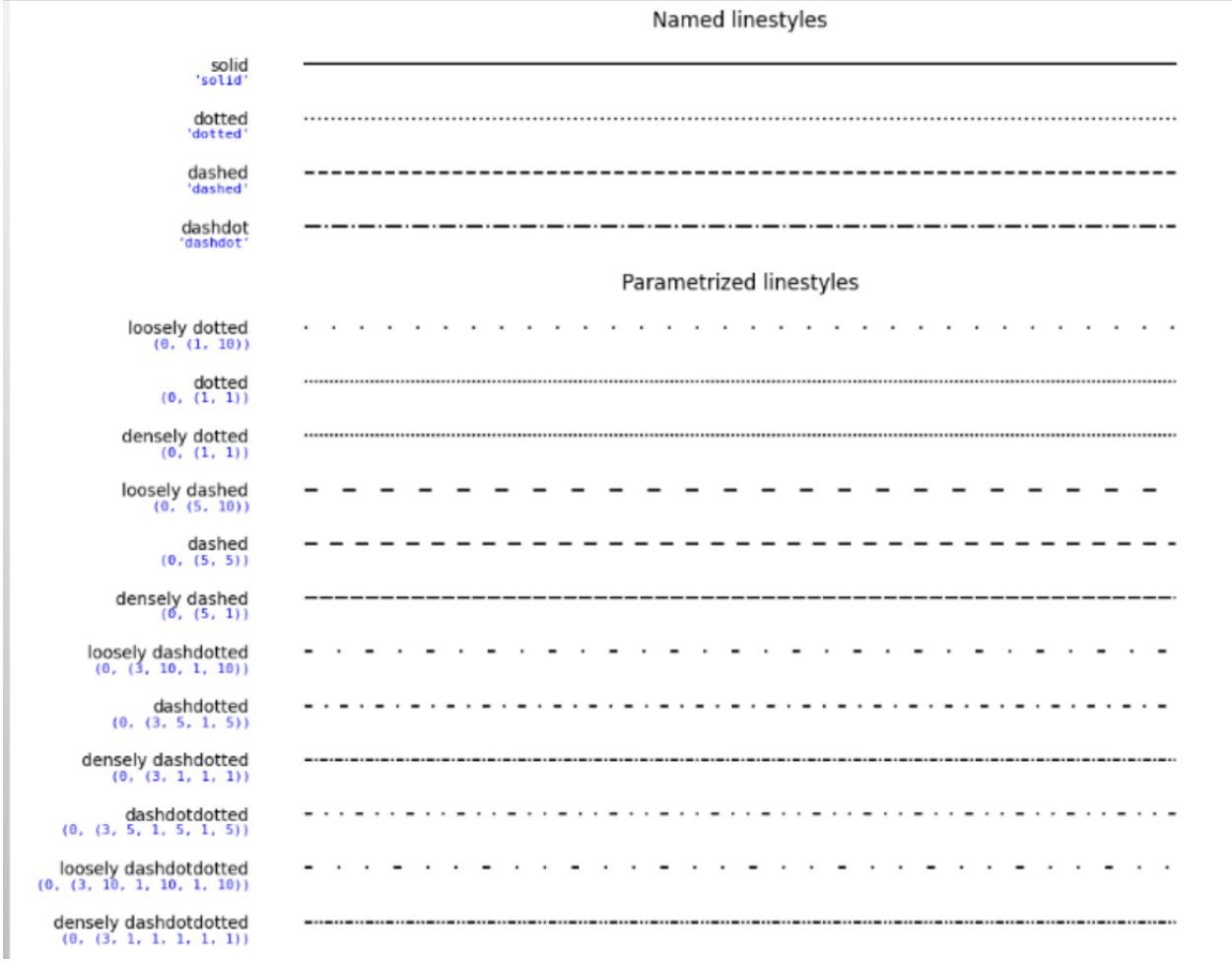


Plotting data from a file

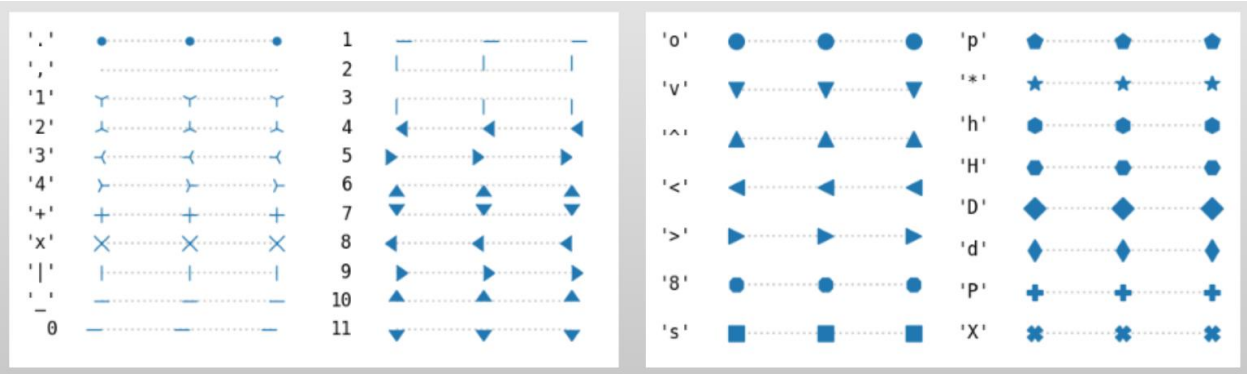
```
>> import matplotlib.•yplot as •1t
>> import numpy as np
>> data = np.genfromtxt("file.csv", delimiter=",", names=["x", "y"])
>> plt.plot(data['x'], data["])
```



Linestyles



Markers

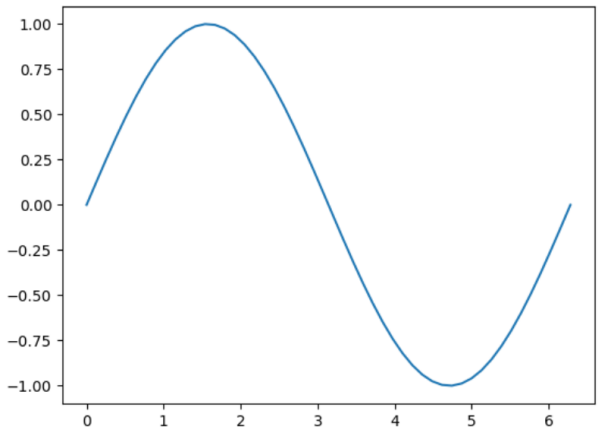


Matplotlib Showcase

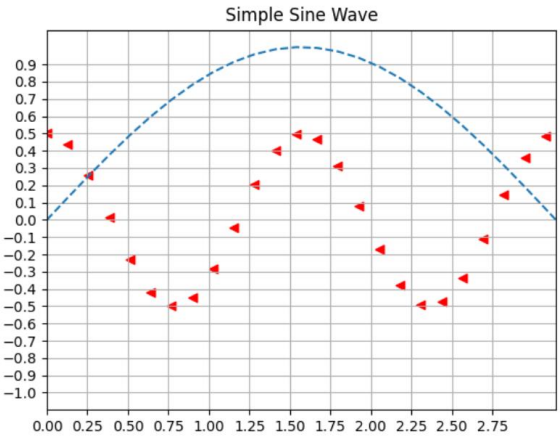
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi)
y = np.sin(x)

plt.plot(x, y)
plt.show()
```




```
plt.plot(x, y, linestyle = '--')
plt.title('Simple Sine Wave')
plt.xlim(0, np.pi)
y2 = np.cos(2*x) ** 2 - 0.5
plt.scatter(x, y2, marker='<', c = 'r')
plt.xticks(np.arange(0, 3, 0.25))
plt.yticks(np.arange(-1, 1, 0.1))
plt.grid()
```



Subplots

```
import numpy as np
import matplotlib.pyplot as plt

x1 = np.linspace(0.0, 5.0)
x2 = np.linspace(0.0, 2.0)

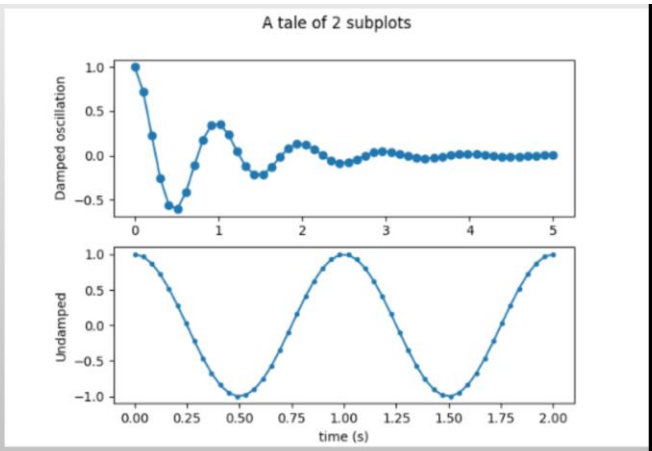
y1 = np.cos(2 * np.pi * x1) * np.exp(-x1)
y2 = np.cos(2 * np.pi * x2)

fig, (ax1, ax2) = plt.subplots(2, 1)
fig.suptitle('A tale of 2 subplots')

ax1.plot(x1, y1, 'o-')
ax1.set_ylabel('Damped oscillation')

ax2.plot(x2, y2, '.-')
ax2.set_xlabel('time (s)')
ax2.set_ylabel('Undamped')

plt.show()
```



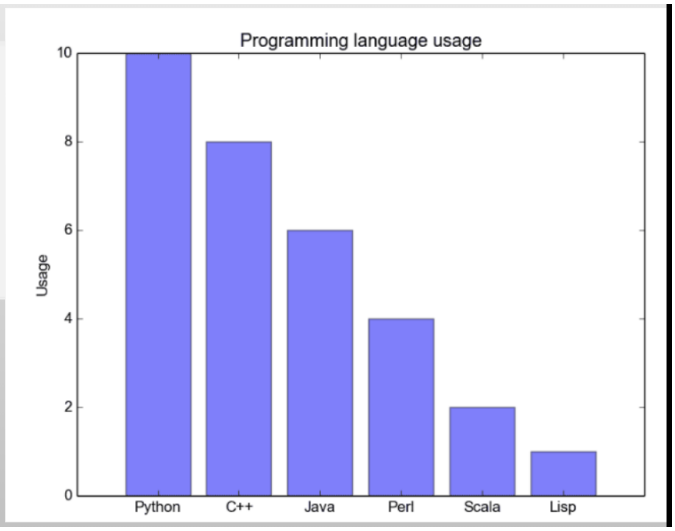
Bar chart

```
import matplotlib.pyplot as plt; plt.rcdefaults()
import numpy as np
import matplotlib.pyplot as plt

objects = ('Python', 'C++', 'Java', 'Perl', 'Scala', 'Lisp')
y_pos = np.arange(len(objects))
performance = [10,8,6,4,2,1]

plt.bar(y_pos, performance, align='center', alpha=0.5)
plt.xticks(y_pos, objects)
plt.ylabel('Usage')
plt.title('Programming language usage')

plt.show()
```



Stacked bar chart

```
import matplotlib.pyplot as plt

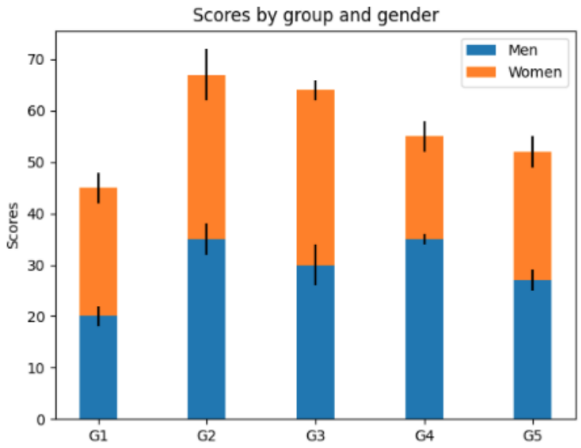

labels = ['G1', 'G2', 'G3', 'G4', 'G5']
men_means = [20, 35, 30, 35, 27]
women_means = [25, 32, 34, 20, 25]
men_std = [2, 3, 4, 1, 2]
women_std = [3, 5, 2, 3, 3]
width = 0.35       # the width of the bars: can also be len(x) sequence

fig, ax = plt.subplots()

ax.bar(labels, men_means, width, yerr=men_std, label='Men')
ax.bar(labels, women_means, width, yerr=women_std, bottom=men_means,
       label='Women')

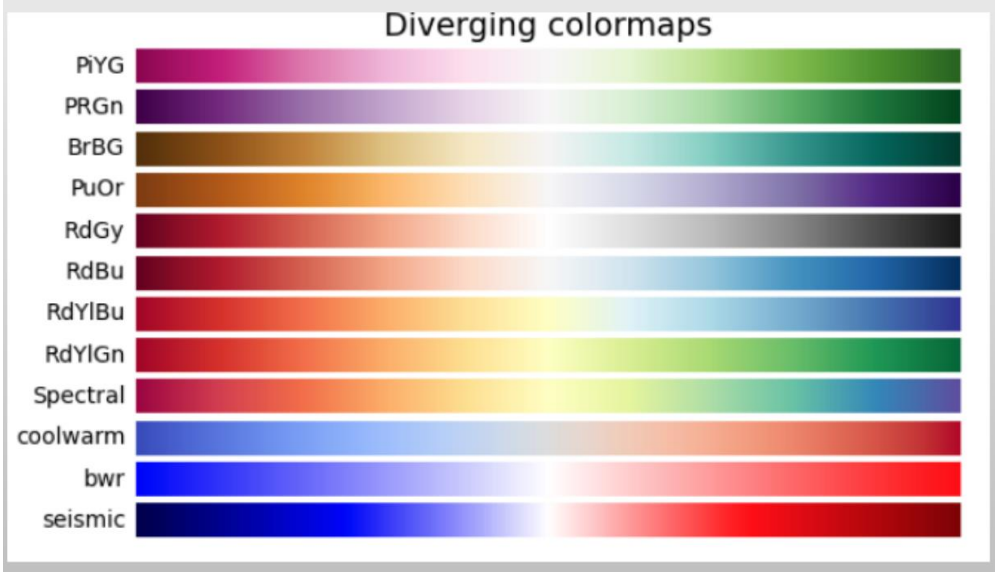
ax.set_ylabel('Scores')
ax.set_title('Scores by group and gender')
ax.legend()

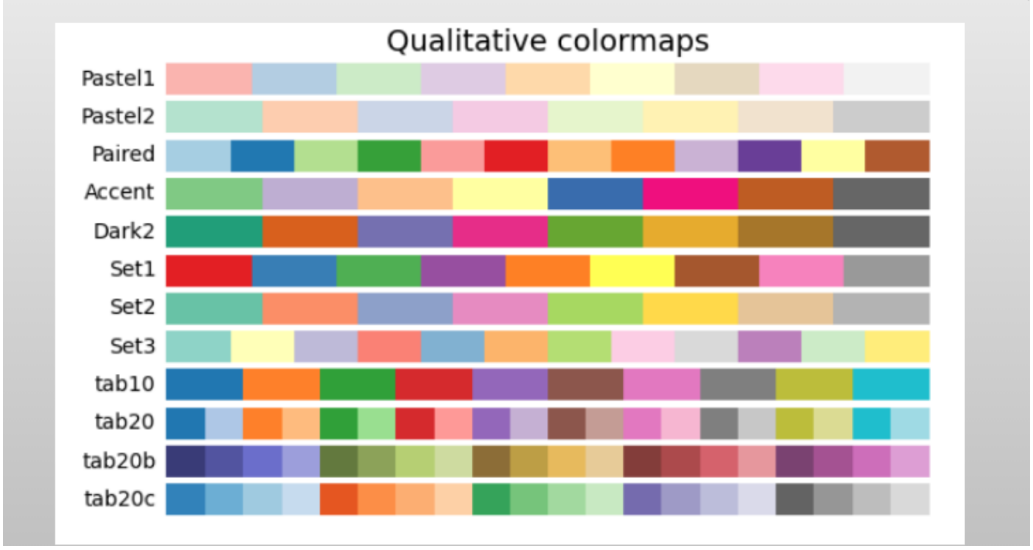
plt.show()
```



Colormaps



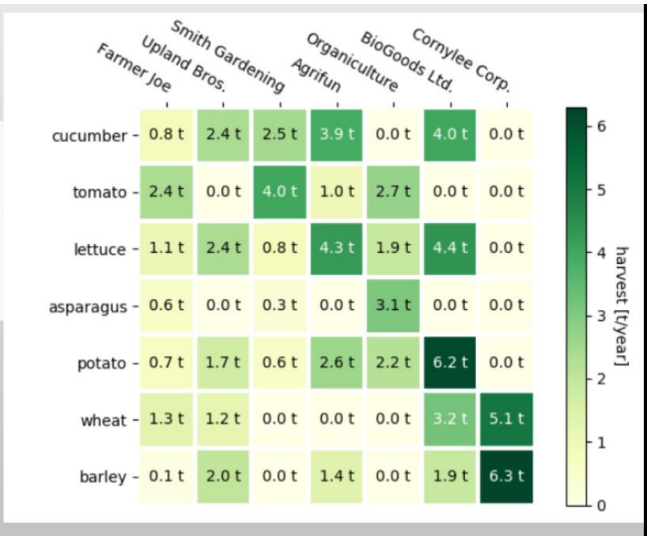




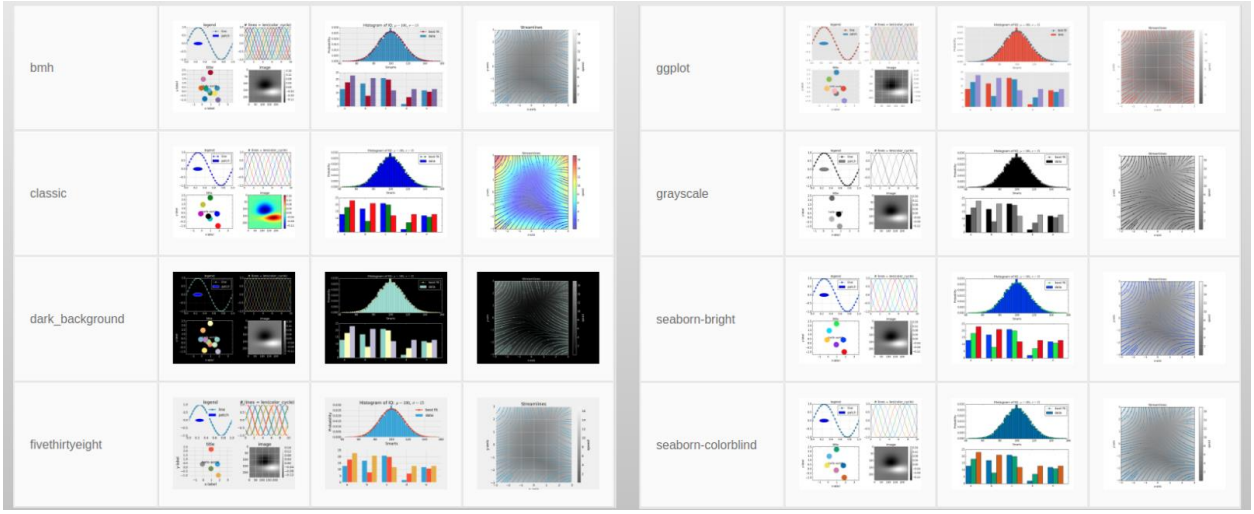

```
fig, ax = plt.subplots()

im, cbar = heatmap(harvest, vegetables, farmers, ax=ax,
                   cmap="YlGn", cbarlabel="harvest [t/year]")
texts = annotate_heatmap(im, valfmt="{x:.1f} t")

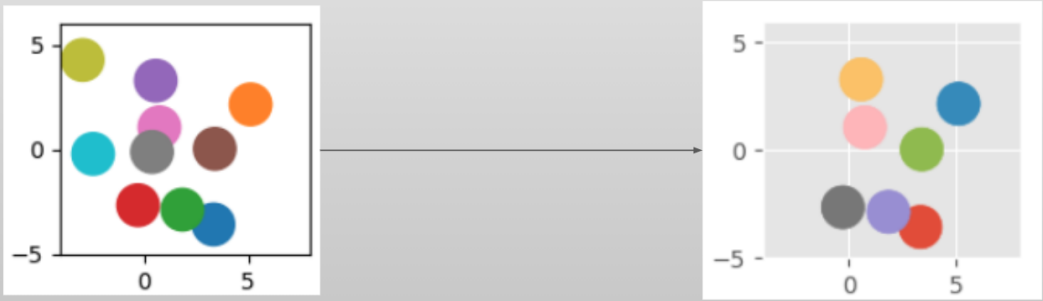
fig.tight_layout()
plt.show()
```



Styles




```
>> plt.style.use('ggplot')
```



Xkcd Style

```
with plt.xkcd():
    # Based on "Stove Ownership" from XKCD by Randall Munroe
    # https://xkcd.com/418/

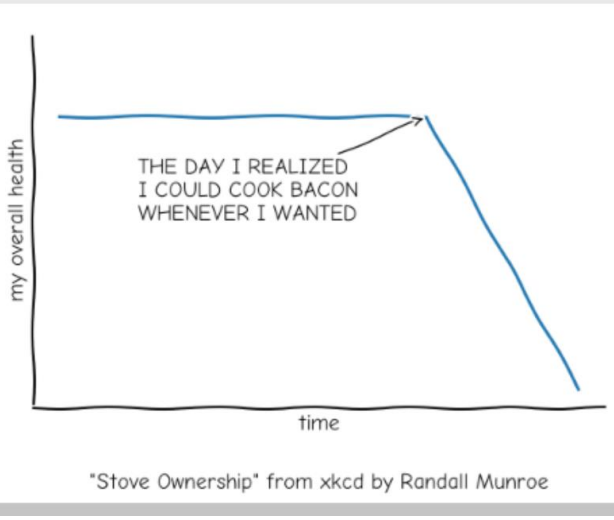
    fig = plt.figure()
    ax = fig.add_axes((0.1, 0.2, 0.8, 0.7))
    ax.spines['right'].set_color('none')
    ax.spines['top'].set_color('none')
    ax.set_xticks([])
    ax.set_yticks([])
    ax.set_ylim([-30, 10])

    data = np.ones(100)
    data[70:] -= np.arange(30)

    ax.annotate(
        'THE DAY I REALIZED\nI COULD COOK BACON\nWHENEVER I WANTED',
        xy=(70, 1), arrowprops=dict(arrowstyle='->'), xytext=(15, -10))

    ax.plot(data)

    ax.set_xlabel('time')
    ax.set_ylabel('my overall health')
    fig.text(
        0.5, 0.05,
        '"Stove Ownership" from xkcd by Randall Munroe',
        ha='center')
```



Some cool plots

Lorentz Attractor

```python
def lorenz(x, y, z, s=10, r=28, b=2.667):
    """
    Given:
       x, y, z: a point of interest in three dimensional space
       s, r, b: parameters defining the lorenz attractor
    Returns:
       x_dot, y_dot, z_dot: values of the lorenz attractor's partial
           derivatives at the point x, y, z
    """
    x_dot = s*(y - x)
    y_dot = r*x - y - x*z
    z_dot = x*y - b*z
    return x_dot, y_dot, z_dot


dt = 0.01
num_steps = 10000

# Need one more for the initial values
xs = np.empty(num_steps + 1)
ys = np.empty(num_steps + 1)
zs = np.empty(num_steps + 1)

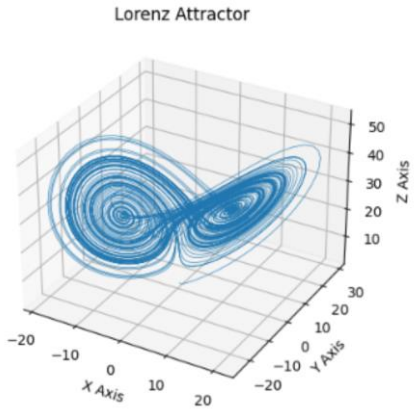
# Set initial values
xs[0], ys[0], zs[0] = (0., 1., 1.05)

# Step through "time", calculating the partial derivatives at the current point
# and using them to estimate the next point
for i in range(num_steps):
    x_dot, y_dot, z_dot = lorenz(xs[i], ys[i], zs[i])
    xs[i + 1] = xs[i] + (x_dot * dt)
    ys[i + 1] = ys[i] + (y_dot * dt)
    zs[i + 1] = zs[i] + (z_dot * dt)


# Plot
fig = plt.figure()
ax = fig.gca(projection='3d')

ax.plot(xs, ys, zs, lw=0.5)
ax.set_xlabel("X Axis")
ax.set_ylabel("Y Axis")
ax.set_zlabel("Z Axis")
ax.set_title("Lorenz Attractor")

plt.show()
```



3D Surface with polar coordinates

```python
import matplotlib.pyplot as plt
import numpy as np


fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

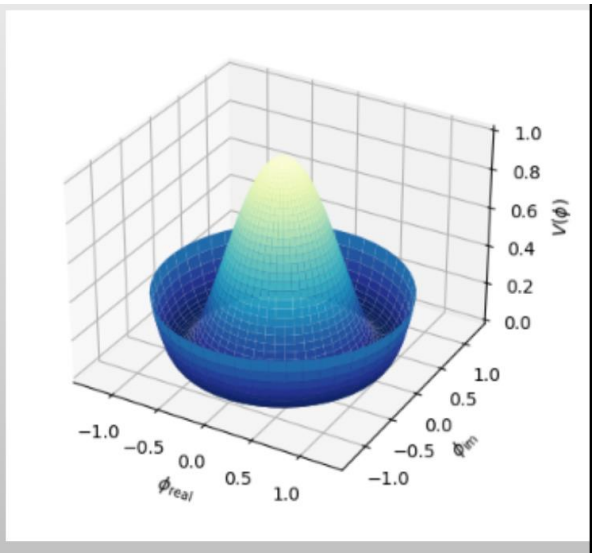
# Create the mesh in polar coordinates and compute corresponding Z.
r = np.linspace(0, 1.25, 50)
p = np.linspace(0, 2*np.pi, 50)
R, P = np.meshgrid(r, p)
Z = ((R**2 - 1)**2)

# Express the mesh in the cartesian system.
X, Y = R*np.cos(P), R*np.sin(P)

# Plot the surface.
ax.plot_surface(X, Y, Z, cmap=plt.cm.YlGnBu_r)

# Tweak the limits and add latex math labels.
ax.set_zlim(0, 1)
ax.set_xlabel(r'$\phi_\mathrm{real}$')
ax.set_ylabel(r'$\phi_\mathrm{im}$')
ax.set_zlabel(r'$V(\phi)$')

plt.show()
```



Costum hillshading in a 3D surface plot

```
from matplotlib import cbook
from matplotlib import cm
from matplotlib.colors import LightSource
import matplotlib.pyplot as plt
import numpy as np

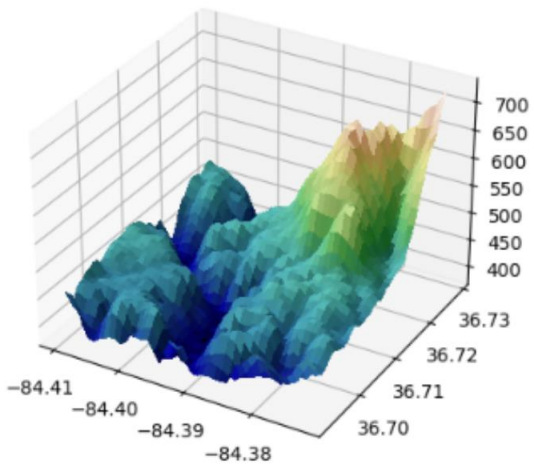
# Load and format data
dem = cbook.get_sample_data('jacksboro_fault_dem.npz', np_load=True)
z = dem['elevation']
nrows, ncols = z.shape
x = np.linspace(dem['xmin'], dem['xmax'], ncols)
y = np.linspace(dem['ymin'], dem['ymax'], nrows)
x, y = np.meshgrid(x, y)

region = np.s_[5:50, 5:50]
x, y, z = x[region], y[region], z[region]

# Set up plot
fig, ax = plt.subplots(subplot_kw=dict(projection='3d'))

ls = LightSource(270, 45)
# To use a custom hillshading mode, override the built-in shading and pass
# in the rgb colors of the shaded surface calculated from "shade".
rgb = ls.shade(z, cmap=cm.gist_earth, vert_exag=0.1, blend_mode='soft')
surf = ax.plot_surface(x, y, z, rstride=1, cstride=1, facecolors=rgb,
                       linewidth=0, antialiased=False, shade=False)

plt.show()
```



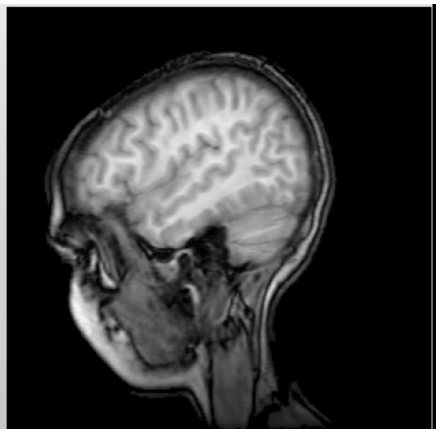
MRI Visualization

```
import matplotlib.pyplot as plt
import matplotlib.cbook as cbook
import numpy as np

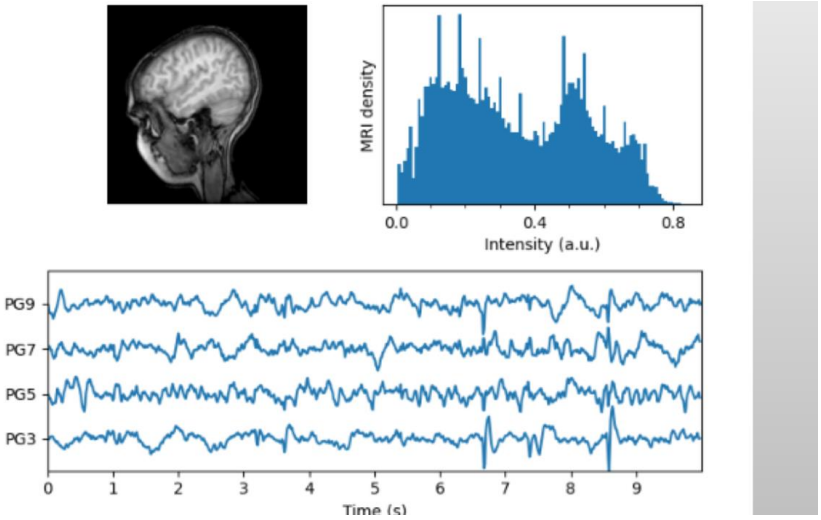

# Data are 256x256 16 bit integers.
with cbook.get_sample_data('s1045.ima.gz') as dfile:
    im = np.frombuffer(dfile.read(), np.uint16).reshape((256, 256))

fig, ax = plt.subplots(num="MRI_demo")
ax.imshow(im, cmap="gray")
ax.axis('off')

plt.show()
```



MRI Visualization with EEG



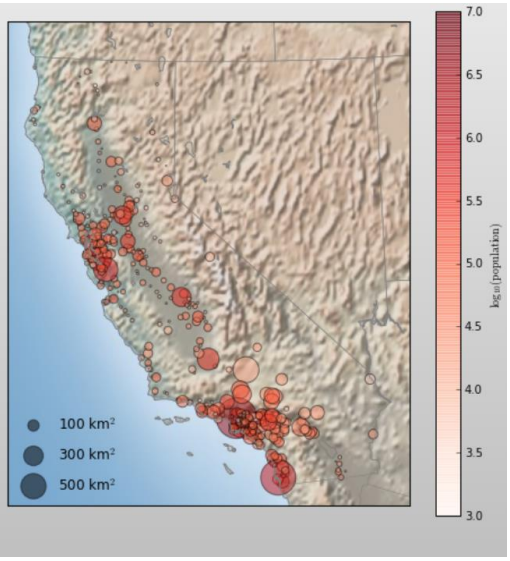
Plotting on a map with Basemap

```
# 1. Draw the map background
fig = plt.figure(figsize=(8, 8))
m = Basemap(projection='lcc', resolution='h',
            lat_0=37.5, lon_0=-119,
            width=1E6, height=1.2E6)
m.shadedrelief()
m.drawcoastlines(color='gray')
m.drawcountries(color='gray')
m.drawstates(color='gray')

# 2. scatter city data, with color reflecting population
# and size reflecting area
m.scatter(lon, lat, latlon=True,
          c=np.log10(population), s=area,
          cmap='Reds', alpha=0.5)

# 3. create colorbar and legend
plt.colorbar(label=r'$\log_{10}({\rm population})$')
plt.clim(3, 7)

# make legend with dummy points
for a in [100, 300, 500]:
    plt.scatter([], [], c='k', alpha=0.5, s=a,
                label=str(a) + ' km$^2$')
plt.legend(scatterpoints=1, frameon=False,
           labelspacing=1, loc='lower left');
```



Conclusion

- Always consider the guidelines for good graphs (most importantly -easy to read / understand, self-contained).
- The lecture presented some of the basic utility of matplotlib, but this is just scratching the surface.
- Any 2D or 3D plots you have seen can be rendered in very high quality using matplotlib.